Machine Learning Steps 5 & 6

5. Perform hyperparameter tuning on the best model to optimize it for the problem

What it is

Hyperparameter tuning is the process of optimizing model parameters that control the learning process. These parameters are not learned from data but must be set before training. In the FURSAN framework, hyperparameter tuning was applied to both the Random Forest regression model and the XGBoost classification model to improve prediction performance.

Why is it important

Hyperparameter tuning improves model performance by:

- Increasing prediction accuracy
- Reducing overfitting
- Improving generalisation to unseen data
- Optimising model efficiency
- Enhancing recall for high-risk satellite conjunction events

Since FURSAN prioritises safety-critical predictions, the model was optimised using the F2-score, which emphasises recall and ensures high-risk collision events are not missed.

Key Steps

1. Define hyperparameter search space
2. Select evaluation metric (F2-score)
3. Use RandomizedSearchCV for tuning
4. Train models using cross-validation
5. Select best hyperparameters
6. Train final optimized model

Code used:

Code Section 1: Library Imports and Setup

```
import pandas as pd
import numpy as np
import time
import joblib
import os

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import (
    fbeta_score, make_scorer,
    accuracy_score, precision_score, recall_score, f1_score
)

from xgboost import XGBClassifier
```

Figure 5.1: Library Imports

Code Section 2: Data Loading and Overview

```python
# ------------------------------
# DATA INGESTION
# ------------------------------
train_data = pd.read_csv("../data/raw/train_data.csv")
test_data  = pd.read_csv("../data/raw/test_data.csv")

print("Train shape:", train_data.shape)
print("Test shape :", test_data.shape)
```

Figure 5.2: Data Loading Code

Code Section 3: Target Variable Extraction

The target variable is extracted from both training and testing datasets. The system checks whether the dataset contains a score or risk column and selects the appropriate variable. The remaining columns are used as input features for model training.

```python
# ------------------------------
# FEATURE ENGINEERING
# (Mahalanobis, covariance normalization, SMOTE done prior)
# ------------------------------

target_train = 'score' if 'score' in train_data.columns else 'risk'
target_test  = 'score' if 'score' in test_data.columns else 'risk'

y_train = train_data[target_train]
y_test  = test_data[target_test]

X_train = train_data.drop(columns=[target_train])
X_test  = test_data.drop(columns=[target_test])
```

Figure 5.3: Target Variable Extraction

Code Section 4: Remove Identifier Columns

Non-informative identifier columns such as event IDs and mission IDs are removed from the dataset. This prevents data leakage and improves model generalisation.

```python
# Remove ID columns
id_cols = [col for col in X_train.columns if 'id' in col.lower()]
X_train = X_train.drop(columns=id_cols, errors='ignore')
X_test  = X_test.drop(columns=id_cols, errors='ignore')

print("Removed ID columns:", id_cols)
```

Figure 5.4: Remove ID Columns

Code Section 5: Multi-Class Risk Label Creation

A function is defined to convert continuous risk scores into multi-class labels. The data is divided into three categories: high risk, medium risk, and low risk. This enables classification-based risk prediction.

```python
# ------------------------------
# Create Multi-Class Risk Labels
# ------------------------------
def create_risk_class(y_log):
    conditions = [
        y_log > -4,                          # High Risk
        (y_log <= -4) & (y_log > -6),        # Medium Risk
        y_log <= -6                          # Low Risk
    ]
    return np.select(conditions, [0, 1, 2]).astype(int)

y_train_class = create_risk_class(y_train)
y_test_class  = create_risk_class(y_test)

print("Class distribution:\n", pd.Series(y_train_class).value_counts())
```

Figure 5.5: Risk Class Creation

Code Section 6: Categorical Feature Encoding

Categorical features are converted into numeric values using factorisation. This step ensures compatibility with machine learning models.

```python
# ------------------------------
# Encode categorical features
# ------------------------------
combined = pd.concat([X_train, X_test], axis=0)

for col in combined.select_dtypes(include=['object', 'string']).columns:
    combined[col] = combined[col].astype(str)
    combined[col] = combined[col].factorize()[0]

X_train = combined.iloc[:len(X_train), :]
X_test  = combined.iloc[len(X_train):, :]
```

Figure 5.6: Feature Encoding

Code Section 7: Data Cleaning and Pre-processing

Missing values, infinite values, and outliers are handled. Median imputation is applied to missing values, and extreme values are clipped to maintain numerical stability.

```python
# ------------------------------
# Data Cleaning
# ------------------------------
X_train = X_train.replace([np.inf, -np.inf], np.nan)
X_test  = X_test.replace([np.inf, -np.inf], np.nan)

X_train = X_train.fillna(X_train.median())
X_test  = X_test.fillna(X_test.median())

X_train = X_train.clip(-1e6, 1e6)
X_test  = X_test.clip(-1e6, 1e6)
```

Figure 5.7: Data Cleaning

Code Section 8: F2 Score Metric Definition

The F2 score metric is defined to prioritise recall. This ensures that high-risk satellite collision events are not missed during prediction.

```python
# F2 scorer (recall-focused)
f2_scorer = make_scorer(fbeta_score, beta=2, average='weighted')
```

Figure 5.8: F2 Score Definition

Code Section 9: Random Forest Model Definition, Hyperparameter Search Space and using RandomizedSearchCV for Random Forest and Random Forest Model Training

The Random Forest regression model is defined as the first stage of the hybrid architecture.

The hyperparameter search space is defined for the Random Forest model. This includes parameters such as number of trees, tree depth, and minimum samples per split.

RandomizedSearchCV is used to search for optimal hyperparameters using cross-validation.

```
# ------------------------------
# Random Forest (Regression Stage)
# ------------------------------
print("\n RF tuning...")

sample_size = min(50000, len(X_train))
idx = np.random.choice(len(X_train), sample_size, replace=False)

X_sample = X_train.iloc[idx]
y_sample = y_train.iloc[idx]

rf = RandomForestRegressor(random_state=42, n_jobs=-1)

rf_param_dist = {
    "n_estimators": [50, 100],
    "max_depth": [10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

rf_random = RandomizedSearchCV(
    rf,
    rf_param_dist,
    n_iter=5,
    cv=2,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

start = time.time()
rf_random.fit(X_sample, y_sample)
end = time.time()
```

```
best_rf = rf_random.best_estimator_

print("✅ RF Best Params:", rf_random.best_params_)
print("⏱ RF Time:", round(end - start, 2), "sec")
```

Figure 5.9: Random Forest Model

The Random Forest model is trained using a sampled subset of data to reduce computational time.

Code Section 10: Hybrid Feature Creation

Random Forest predictions are appended to the dataset to create hybrid features.

```
# ------------------------------
# Hybrid Feature Creation
# ------------------------------
rf_train_pred = best_rf.predict(X_train)
rf_test_pred  = best_rf.predict(X_test)

X_train_hybrid = np.column_stack((X_train, rf_train_pred))
X_test_hybrid  = np.column_stack((X_test, rf_test_pred))
```

Figure 5.10: Hybrid Feature Creation

Code Section 11: XGBoost Model Definition, Hyperparameter Search Space, using RandomizedSearchCV for XGBoost and XGBoost Model Training

The XGBoost classifier is defined as the second stage of the hybrid architecture.

Hyperparameter ranges for the XGBoost classifier are defined to optimise performance.

RandomizedSearchCV is applied to identify the best hyperparameters for XGBoost.

```
# XGBoost (Classification Stage)
# ------------------------------
print("\n XGBoost tuning...")

xgb = XGBClassifier(
    objective='multi:softprob',
    num_class=3,
    eval_metric='mlogloss',
    tree_method='hist',
    random_state=42,
    n_jobs=-1
)

xgb_param_dist = {
    "n_estimators": [100, 150],
    "max_depth": [3, 5],
    "learning_rate": [0.05, 0.1],
    "subsample": [0.8],
    "colsample_bytree": [0.8]
}

xgb_random = RandomizedSearchCV(
    xgb,
    xgb_param_dist,
    n_iter=5,
    scoring=f2_scorer,
    cv=2,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

start = time.time()
xgb_random.fit(X_train_hybrid, y_train_class)
end = time.time()
```

```
best_xgb = xgb_random.best_estimator_

print("✅ XGB Best Params:", xgb_random.best_params_)
print("⏱ XGB Time:", round(end - start, 2), "sec")
```

Figure 5.11: XGBoost Model Definition

Code Section 12: Final Model Evaluation

The model is evaluated using accuracy, precision, recall, F1 score, and F2 score.

```
# ====================================================
# FINAL EVALUATION ON TEST DATASET
# ====================================================

print("\n📊 Final Evaluation...")

y_pred = best_xgb.predict(X_test_hybrid)

accuracy  = accuracy_score(y_test_class, y_pred)
precision = precision_score(y_test_class, y_pred, average='weighted')
recall    = recall_score(y_test_class, y_pred, average='weighted')
f1        = f1_score(y_test_class, y_pred, average='weighted')
f2        = fbeta_score(y_test_class, y_pred, beta=2, average='weighted')

print("\n===== FINAL RESULTS =====")
print("Accuracy :", round(accuracy, 6))
print("Precision:", round(precision, 6))
print("Recall   :", round(recall, 6))
print("F1 Score :", round(f1, 6))
print("F2 Score :", round(f2, 6))
```

Figure 5.12: Final Evaluation Metrics

Code Section 13: Save Hybrid Model

The trained hybrid model is saved using Joblib.

```
# ====================================================
# SAVE MODEL (PKL)
# ====================================================

os.makedirs("../models", exist_ok=True)

joblib.dump({
    "rf_model": best_rf,
    "xgb_model": best_xgb
}, "../models/hybrid_model.pkl")

print("✅ Model saved as hybrid_model.pkl")
```

Figure 5.13: Model Saving

Code Section 14: Save Risk Output Files

Predicted risk categories are separated into high, medium, and low risk files.

```python
# ====================================================
# SAVE RISK OUTPUT FILES
# ====================================================

results = test_data.copy()
results["predicted_class"] = y_pred

# Map classes
risk_map = {
    0: "high",
    1: "medium",
    2: "low"
}

results["risk_level"] = results["predicted_class"].map(risk_map)

# Split datasets
highriskcsv   = results[results["risk_level"] == "high"]
mediumriskcsv = results[results["risk_level"] == "medium"]
lowriskcsv    = results[results["risk_level"] == "low"]

# Save CSVs
os.makedirs("../outputs", exist_ok=True)

highriskcsv.to_csv("../outputs/high_risk.csv", index=False)
mediumriskcsv.to_csv("../outputs/medium_risk.csv", index=False)
lowriskcsv.to_csv("../outputs/low_risk.csv", index=False)

print("✅ Risk CSVs saved (high, medium, low)")
```

Figure 5.14: Saving the Risk Output Files

6. Evaluate the best model on the testing set

Code Section 1: Load Test Dataset

The test dataset is loaded for evaluation.

```python
# ====================================================
# 1. LOAD TEST DATA
# ====================================================

test_data = pd.read_csv("../data/raw/test_data.csv")
print("Test shape:", test_data.shape)
```

Figure 6.1: Load Test Dataset

Code Section 2: Load Hybrid Model

The saved hybrid model is loaded from disk.

```python
# ====================================================
# 2. LOAD MODEL
# ====================================================

model = joblib.load("../models/hybrid_model.pkl")

rf_model  = model["rf_model"]
xgb_model = model["xgb_model"]

print("✅ Hybrid model loaded")
```

Figure 6.2: Load Hybrid Model

Code Section 3: Target Variable Extraction

The target variable is separated from input features.

```
# ====================================================
# 3. EXTRACT TARGET
# ====================================================

target_test = 'score' if 'score' in test_data.columns else 'risk'

y_test = test_data[target_test]
X_test = test_data.drop(columns=[target_test])
```

Figure 6.3: Target Extraction

Code Section 4: Remove ID Columns

Identifier columns are removed from the dataset.

```
# ====================================================
# 4. REMOVE ID COLUMNS
# ====================================================

id_cols = [col for col in X_test.columns if 'id' in col.lower()]
X_test = X_test.drop(columns=id_cols, errors='ignore')

print("Removed ID columns:", id_cols)
```

Figure 6.4: Remove ID Columns

Code Section 5: Multi-Class Risk Creation

Risk categories are created from test data.

```
# ====================================================
# 5. CREATE MULTI-CLASS TARGET
# ====================================================

def create_risk_class(y_log):
    conditions = [
        y_log > -4,                          # High Risk
        (y_log <= -4) & (y_log > -6),        # Medium Risk
        y_log <= -6                          # Low Risk
    ]
    return np.select(conditions, [0, 1, 2]).astype(int)

y_test_class = create_risk_class(y_test)
```

Figure 6.5: Risk Class Creation

Code Section 6: Data Cleaning and Encoding

The test dataset is cleaned and encoded.

```
# ====================================================
# 6. HANDLE CATEGORICAL + CLEAN DATA
# ====================================================

for col in X_test.select_dtypes(include=['object', 'string']).columns:
    X_test[col] = X_test[col].astype(str)
    X_test[col] = X_test[col].factorize()[0]

X_test = X_test.replace([np.inf, -np.inf], np.nan)
X_test = X_test.fillna(X_test.median())
X_test = X_test.clip(-1e6, 1e6)

print("✅ Test data processed")
```

Figure 6.6: Data Cleaning

Code Section 7: Random Forest Prediction, Hybrid Feature Creation and XGBoost Classification

```
# ====================================================
# 7. HYBRID PREDICTION PIPELINE
# ====================================================

# Stage 1: Random Forest Regression
rf_pred = rf_model.predict(X_test)

# Stage 2: Hybrid Features
X_test_hybrid = np.column_stack((X_test, rf_pred))

# Stage 3: XGBoost Classification
y_pred = xgb_model.predict(X_test_hybrid)
```

Figure 6.7: Random Forest Prediction

Code Section 10: Evaluation Metrics

Performance metrics are calculated.

```
# ====================================================
# 8. EVALUATION METRICS
# ====================================================

accuracy  = accuracy_score(y_test_class, y_pred)
precision = precision_score(y_test_class, y_pred, average='weighted')
recall    = recall_score(y_test_class, y_pred, average='weighted')
f1        = f1_score(y_test_class, y_pred, average='weighted')
f2        = fbeta_score(y_test_class, y_pred, beta=2, average='weighted')

print("\n===== FINAL TEST RESULTS =====")
print("Accuracy :", round(accuracy, 6))
print("Precision:", round(precision, 6))
print("Recall   :", round(recall, 6))
print("F1 Score :", round(f1, 6))
print("F2 Score :", round(f2, 6))
```

Figure 6.10: Evaluation Metrics

Code Section 11: Classification Report

A classification report is generated.

```
# ====================================================
# 9. CLASSIFICATION REPORT
# ====================================================

print("\n===== CLASSIFICATION REPORT =====")
print(classification_report(y_test_class, y_pred))
```

Figure 6.11: Classification Report

Code Section 12: Save Risk Outputs

Risk output files are generated.

```python
# ====================================================
# 10. SAVE RISK OUTPUT FILES
# ====================================================

results = test_data.copy()
results["predicted_class"] = y_pred

risk_map = {
    0: "high",
    1: "medium",
    2: "low"
}

results["risk_level"] = results["predicted_class"].map(risk_map)

highriskcsv   = results[results["risk_level"] == "high"]
mediumriskcsv = results[results["risk_level"] == "medium"]
lowriskcsv    = results[results["risk_level"] == "low"]

os.makedirs("../outputs", exist_ok=True)

highriskcsv.to_csv("../outputs/high_risk_test_output.csv", index=False)
mediumriskcsv.to_csv("../outputs/medium_risk_test_output.csv", index=False)
lowriskcsv.to_csv("../outputs/low_risk_test_output.csv", index=False)

print("✅ Risk CSVs saved")
```

Figure 6.12: Saving the Final Risk Output Files

Code Section 13: Confusion Matrix

A confusion matrix is generated for performance visualisation.

```python
# 11. CONFUSION MATRIX
# ====================================================

cm = confusion_matrix(y_test_class, y_pred)

# Custom color map: Green → Orange → Red
cmap = LinearSegmentedColormap.from_list(
    "risk_map",
    [🟥"#c0392b", 🟧"#f39c12", 🟩"#27ae60"]
)

plt.figure(figsize=(6,5))
plt.imshow(cm, interpolation='nearest', cmap=cmap)

plt.title("FURSAN Confusion Matrix (LEO Risk Levels)", fontweight='bold')
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.colorbar()

classes = ["High", "Medium", "Low"]

plt.xticks(range(len(classes)), classes)
plt.yticks(range(len(classes)), classes)

# Add values inside matrix
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j],
                 ha='center',
                 va='center',
                 fontweight='bold')

plt.tight_layout()
plt.show()
```

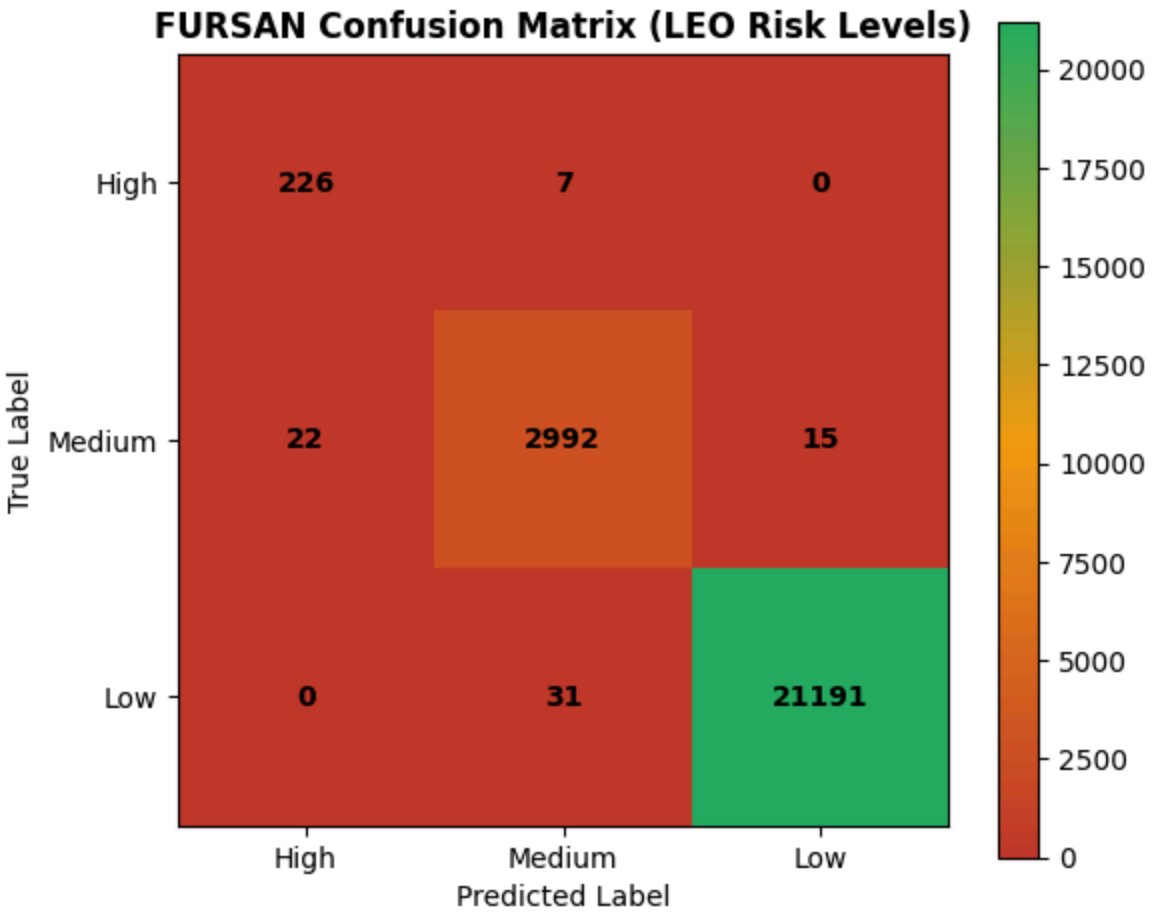

Figure 6.13: Confusion Matrix